

Parallel Mosaic Recreation

This project recreates an image using a mosaic of smaller images by matching the average colors. It uses parallel processing to speed up the mosaic recreation and loading the sample images process.

Implementation

Color Matching:

First I had to find the average color of each block by finding the average color of red, green, and blue for both the image and the sample images.. Then once the average color is found I used the squared Euclidean distance formula to find the best matching image to replace the block.

Parallel Processing:

To improve performance I used Future. Each block is processed concurrently, where a separate Future handles the selection and placement of the best matching image. In terms of loading in the sample images, I divided the loading process into batches to prevent memory overflow.

Additionally using Future to load in the images concurrently also speeds up the process. Once the best matching sample image is found the sample image will be drawn into the block concurrently.

Caching Loaded Sample Images:

To avoid loading the sample images every time the program runs, I used a caching system. A cache file (image_cache.dat) is used to store the precomputed average colors of the sample images. Once the program runs it will check if the cache exists, if not then it will proceed to load

and compute the average colors of the sample images. If the cache does exist then it will use the precomputed data instead of doing the same process again. However, if the cache is outdated then the process will process the sample images and update the cache.

Mosaic Creation:

The process involves dividing the input image into small blocks of a specified size. Then it extracts the average color of each block and finds the best matching sample image using the squared Euclidean distance formula. Once the best matching image is found it draws the image into the block that has the best matching color. This of course uses Future to ensure the process is faster.

Challenges and Optimization

The first challenge was trying to avoid over-thread or over use of memory which causes the program to become slower when running in parallel. When I first implement the parallel program, the process becomes slower. To tackle this I tried to split the process of loading the sample images into smaller batches. This prevents memory overflow, but it still wasn't fast enough.

Loading a large number of sample images makes the program slower. After dividing it into a batch and using Future to load the images concurrently wasn't fast enough, I used caching to store the precomputed average color of the sample images. This allowed the program to avoid doing calculation of the sample images every time it runs. By implementing caching it speeds up the process by about 0.5 seconds.

Results and Performances

The mosaic recreation is a success, but the speed of the parallel is only a small success. The parallel process of creating the mosaic image is a little but slower than the base implementation. This may be because of over threading by using Future. However, after splitting the process of loading the sample images into batches and using cache to store the precomputed average color of the sample images made the process faster. The performance became better and is a bit faster than the base implementation, so it is a small success. The base implementation takes 4.02625725 seconds while the parallel implementation now takes 4.02625725 seconds.

Future Improvements

- Instead of loading all the sample images, use lazy loading instead. So only the images that are needed are loaded, which can reduce memory consumption when working with a large data set (sample images)
- Improve performance by limiting the number of threads in use. This would fix the problem of over threading that might be the problem that causes the program to be slower.
- Include more error handling. Currently if there is an error, most likely the program will not run. For example when a sample image returns null, by improving error handling it will help the user see which file is actually corrupted or not working.

Sources Used

<https://otfried.org/scala/image.html>

<https://otfried.org/scala/drawing.html>

<https://stackoverflow.com/questions/12408431/how-can-i-get-the-average-color-of-an-image>

<https://stackoverflow.com/questions/55264706/is-there-anyway-to-replace-some-of-the-block-data-to-add-an-image>

<https://dev.to/iamsujit/the-journey-to-building-a-color-matching-library-with-euclidean-distance-2g04#:~:text=The%20Role%20of%20Euclidean%20Distance%20in%20Color%20Matching&text=In%20simpler%20terms%2C%20it%20tells,means%20they%20are%20significantly%20different.>